NTIRE 2026 Image Denoising ($\sigma = 50$) Challenge Factsheet
Two-Stage Attention U-Net with Residual Refinement for Image Denoising

Arun Barkhanda
Department of Mathematics, Clarkson University
8 Clarkson Ave, Potsdam, NY 13699
barkhaa@clarkson.edu

1. Team Details

- **Team name:** Variational Vision
- **Team leader:** Arun Barkhanda
- **Address:** Department of Mathematics, Clarkson University, 8 Clarkson Ave, Potsdam, NY 13699
- **Phone:** +1-315-244-4867
- **Email:** barkhaa@clarkson.edu *(CodaBench account: arunbarkhanda@gmail.com)*
- **Team members:** Single-author submission
- **Team website:** N/A
- **Affiliation:** Clarkson University
- **CodaBench username(s):** arunbarkhanda
- **Best validation phase PSNR:** 29.88
- **Code & model:** https://github.com/arunbarkhanda/NTIRE_2026_arun.git

2. Method Details

2.1. General Method Description

We propose a two-stage denoising pipeline for RGB images corrupted by additive white Gaussian noise with standard deviation $\sigma = 50$. In the first stage, an Attention U-Net predicted an initial clean image from the noisy input. In the second stage, a residual refinement network took the base denoised image as input and predicted a corrective residual. We obtained the final restored image by adding the predicted residual to the base output and clipping intensities to the range $[0, 1]$.

The first stage was responsible for the main denoising process, while the second stage focused on correcting remaining artifacts and recovering fine image structures. We performed inference patch-wise with overlap and Gaussian blending for full-resolution restoration. We additionally applied self-ensemble test-time augmentation to the base model. We denote the base model output as $\hat{x}$, and the final restored image as x.

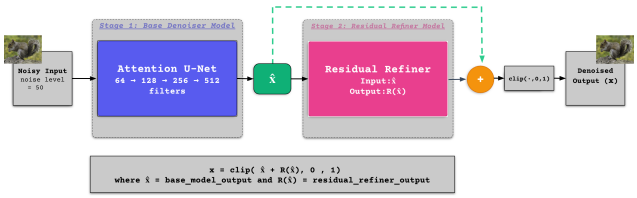

Figure 1. Overview of the two-stage denoising pipeline. The base Attention U-Net produces an initial denoised image $\hat{x}$, which is then refined by the residual network. The final output is computed as $x = \text{clip}(\hat{x} + R(\hat{x}), 0, 1)$.

2.2. Stage 1: Base Attention U-Net

The first-stage denoiser was an Attention U-Net inspired by the U-Net architecture [4] and attention-gated skip connections [3]. The model followed a symmetric encoder–decoder structure with three encoder stages, one bottleneck stage, and three decoder stages.

Encoder Each encoder block contained:
- Two 3×3 convolutional layers
- Batch Normalization after each convolution
- GELU activation
- Dropout (rates: 0.05, 0.05, 0.10 for levels 1–3)
- 2×2 MaxPooling for downsampling

The number of feature channels increased as:

$$64 \rightarrow 128 \rightarrow 256$$

Bottleneck The bottleneck consisted of a convolutional block with 512 filters and dropout rate 0.15.

Decoder Each decoder block included:
- Upsampling by a factor of 2
- Attention-gated skip connection
- Concatenation with attended encoder features
- Two 3×3 convolutional layers with GELU activation

The final layer was a 1×1 convolution with sigmoid activation producing a 3-channel RGB output. We used sigmoid activation in the final layer to constrain the output to $[0, 1]$, suitable for direct image reconstruction. We chose GELU activations for the encoder and decoder blocks as they provided smoother gradients compared to ReLU, which benefited training stability. We used mixed precision training for computational efficiency, while keeping the output layer in float32 for numerical stability.

2.3. Stage 2: Residual Refinement Network

The second-stage model was a lightweight U-Net-style residual refiner. It took the first-stage output $\hat{x}$ as input and predicted a residual correction $R(\hat{x})$. We computed the final refined image as

$$x = \mathrm{clip}\big(\hat{x} + R(\hat{x}), 0, 1\big).$$

The residual refiner used three encoder stages, one bottleneck, and three decoder stages with channel progression

$$32 \to 64 \to 128 \to 256.$$

Each block contained two 3×3 convolutional layers followed by batch normalization and ReLU activation. We used dropout rates of 0.05, 0.05, and 0.10 in the encoder and 0.15 in the bottleneck. The final 1×1 convolution used tanh activation to allow the network to predict both positive and negative residual corrections. During training we embedded the residual network inside a wrapper so that the clipped sum of base output and predicted residual was optimized directly against the clean target image.

2.4. Dataset and Training Strategy

We normalized all images to $[0, 1]$. We added Gaussian noise with $\sigma = 50/255$ to clean patches and clipped noisy inputs to $[0, 1]$.

Base model dataset We trained the base Attention U-Net on the LSDIR dataset [2] using an image-disjoint split (80% train / 20% validation). For each image, we extracted eight 96×96 patches: one center crop, one high-texture crop selected via Laplacian-variance scoring over a 4×4 grid, and six random crops. We augmented training patches with random horizontal/vertical flips and 90° rotations. Validation used five deterministic crops (four corners and center) with fixed-seed noise generation for reproducible PSNR evaluation.

Residual model dataset We trained the residual refiner using cached pairs of base-model outputs and their corresponding clean targets. We first passed noisy patches through the pretrained base model to produce $\hat{x}$ and stored both the base outputs and the corresponding clean patches as compressed `.npz` files to avoid repeated base-model inference.

We mixed residual training samples from the LSDIR dataset and the Describable Textures Dataset (DTD) [1] in an 85%/15% ratio. The additional texture data improved the model's ability to recover high-frequency structures and complex surface patterns during residual refinement. LSDIR contributed four random 96×96 patches per image, while texture samples used center crops. Validation used center crops only. We followed a 90%/10% train–validation split for both datasets.

Training Strategy We trained the base denoiser using the L1 loss (Mean Absolute Error) with the AdamW optimizer. We chose L1 loss as it is less sensitive to outliers and encourages sharper predictions compared to MSE. For AdamW, we used default values ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1 \times 10^{-7}$) with weight decay 1×10^{-5}. We set the initial learning rate to 1×10^{-4} with a batch size of 32 and patch size of 96×96. We trained for up to 50 epochs. We reduced the learning rate by a factor of 0.5 when the validation loss did not improve for 5 consecutive epochs (minimum learning rate 1×10^{-6}), and we applied early stopping with patience 15 based on validation PSNR. We selected the final model according to the highest validation PSNR.

We trained the second-stage residual refiner using MSE loss with the Adam optimizer. We chose MSE for the residual refiner as it penalizes larger errors more strongly, which is beneficial for fine residual corrections. We set the initial learning rate to 1×10^{-4} with a batch size of 64 and patch size 96×96, and we trained for up to 50 epochs. The network predicted a residual correction applied to the base output, and the final result was optimized directly against the clean target image. We applied learning rate reduction on plateau and early stopping based on validation loss.

We trained both models on an NVIDIA A100 40GB SXM4 GPU rented via Lambda Labs cloud platform.

2.5. Inference

At test time, we processed full-resolution images patch-wise using 96×96 patches with overlap 60. We merged overlapping predictions using a Gaussian blending window to reduce visible seams and boundary artifacts.

For the base model, we applied 8-transform self-ensemble test-time augmentation. The transform set consisted of four rotations (0°, 90°, 180°, 270°) and their horizontally flipped counterparts. We denoised each transformed image independently, inverse-transformed back to the original orientation, and averaged to obtain the first-stage prediction.

We passed the averaged base result to the residual refinement model. The residual model predicted an additive correction, and we computed the final restored image by adding

this correction to the base output and clipping intensities to $[0, 1]$.

References

[1] M. Cimpoi, S. Maji, I. Kokkinos, S. Mohamed, and A. Vedaldi. Describing textures in the wild. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014. 2

[2] Yawei Li, Kai Zhang, Jingyun Liang, Jiezhang Cao, Ce Liu, Rui Gong, Yulun Zhang, Hao Tang, Yun Liu, Denis Demandolx, et al. Lsdir: A large scale dataset for image restoration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1775–1787, 2023. 2

[3] Ozan Oktay, Jo Schlemper, Loic Le Folgoc, Matthew Lee, Mattias Heinrich, Kazunari Misawa, Kensaku Mori, Steven McDonagh, Nils Hammerla, Bernhard Kainz, Ben Glocker, and Daniel Rueckert. Attention u-net: Learning where to look for the pancreas. *arXiv preprint arXiv:1804.03999*, 2018. 1

[4] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241. Springer, 2015. 1